

# LIM M2

## Архітектурний довідник процесора

Високодетермінована 16-розрядна мікропрограмна архітектура з чистим мультиплексорним трактом даних та розширеною 24-бітною адресацією.



POINT-TO-POINT DATA ROUTING · ZERO BUS CONTENTION

### OFFICIAL REFERENCE SPECIFICATION

Редакція 0.2 — Жовтень 2026 р.

Специфікація процесора LIM M2. Всі права захищено.

## Contents

|                                                                               |           |
|-------------------------------------------------------------------------------|-----------|
| <b>1 Статус документа та архітектурна концепція</b>                           | <b>3</b>  |
| 1.1 Умовні позначення                                                         | 3         |
| 1.2 Єдина філософія формату інструкцій                                        | 3         |
| <b>2 Архітектурний огляд процесора LIM M2</b>                                 | <b>3</b>  |
| 2.1 Концептуальна схема тракту даних                                          | 5         |
| <b>3 Програмно доступні регістри та стан процесора</b>                        | <b>6</b>  |
| 3.1 Карта регістра прапорців та керування (FREG)                              | 6         |
| <b>4 Формат машинних команд</b>                                               | <b>7</b>  |
| 4.1 Універсальний бітовий шаблон машинної команди                             | 7         |
| 4.2 Клас I: Однобайтові інструкції (1 байт, 0 додаткових октет)               | 7         |
| 4.3 Клас II: Трьохоперандні арифметичні інструкції (2 байти)                  | 7         |
| 4.4 Клас III: Двохоперандні логічні інструкції (2 байти)                      | 8         |
| 4.5 Клас IV: Міжрегістрове переміщення LRR (2 байти)                          | 8         |
| 4.6 Клас V: Апаратний компаратор CMP (2 байти)                                | 8         |
| 4.7 Клас VI: Апаратний циклічний зсув на D-тригерах (BSL, BSR)                | 8         |
| 4.8 Клас VII: Операції зі стеком (PUSH, POP)                                  | 9         |
| 4.9 Клас VIII: Маніпуляція спеціальними регістрами (MSP)                      | 9         |
| 4.10 Клас IX: Інструкції розгалуження та переходів (2–3 байти)                | 9         |
| 4.11 Клас X: Інструкції обміну з пам'яттю LMR та LRM (2–4 байти)              | 10        |
| <b>5 Зведена таблиця команд ядра LIM M2</b>                                   | <b>10</b> |
| <b>6 Детальна специфікація системи команд</b>                                 | <b>11</b> |
| NOP — Апаратна порожня операція та скидання ланцюжка обчислень                | 12        |
| ADD — Додавання з автоматичним каскадуванням переносу                         | 14        |
| SUB — Віднімання з автоматичним каскадуванням позики                          | 16        |
| MUL — Цілочисельне множення зі збереженням старшої частини в регістрі АЛП     | 18        |
| DIV — Цілочисельне ділення зі збереженням залишку у внутрішньому регістрі АЛП | 20        |
| LMR — Завантаження даних із пам'яті та зовнішніх пристроїв у регістри         | 22        |
| LRM — Запис даних із регістрів у пам'ять та зовнішні пристрої                 | 24        |
| LRR — Міжрегістрове переміщення (копіювання з точки Б в точку А)              | 26        |
| CMP — Апаратне порівняння операндів                                           | 28        |
| JMP — Безумовний перехід                                                      | 30        |
| JCR — Умовний перехід за прапорцем переносу                                   | 32        |
| JNZ — Умовний перехід, якщо не нуль                                           | 34        |
| JZ — Умовний перехід за нулем                                                 | 36        |

|                                                                          |           |
|--------------------------------------------------------------------------|-----------|
| AND — Побітове логічне І . . . . .                                       | 38        |
| OR — Побітове логічне АБО . . . . .                                      | 40        |
| NAND — Побітове логічне І-НІ . . . . .                                   | 42        |
| NOR — Побітове логічне АБО-НІ . . . . .                                  | 44        |
| NOT — Побітова інверсія . . . . .                                        | 46        |
| XOR — Побітове виключне АБО . . . . .                                    | 48        |
| XNOR — Побітова еквівалентність . . . . .                                | 50        |
| INC — Інкремент на одиницю . . . . .                                     | 52        |
| DEC — Декремент на одиницю . . . . .                                     | 54        |
| PUSH — Збереження слова в стек . . . . .                                 | 56        |
| POP — Вилучення слова з вершини стека . . . . .                          | 58        |
| BSL — Апаратний зсув ліворуч (ітеративний регістр) . . . . .             | 60        |
| BSR — Апаратний зсув праворуч (ітеративний регістр) . . . . .            | 62        |
| MSP — Маніпуляція спеціальними регістрами (FREG, PTREG) . . . . .        | 64        |
| MOD — Цілочисельний залишок від ділення (modulo) . . . . .               | 66        |
| RSV28 — Апаратне розширення . . . . .                                    | 68        |
| RSV29 — Апаратне розширення . . . . .                                    | 70        |
| RSV30 — Апаратне розширення . . . . .                                    | 72        |
| RSV31 — Апаратне розширення . . . . .                                    | 74        |
| <b>7 Фізичний інтерфейс та цоколівка (34-вивідний корпус)</b>            | <b>76</b> |
| 7.1 Опис системних сигналів . . . . .                                    | 76        |
| <b>8 Підсистема переривань та векторна обробка</b>                       | <b>77</b> |
| 8.1 Єдиний простір 16 векторів переривань . . . . .                      | 77        |
| 8.2 Апаратний протокол квітування (RDI → IA → D0–D7) . . . . .           | 77        |
| 8.3 Збереження контексту у 24-бітному стеку та вхід в обробник . . . . . | 78        |
| 8.4 Повернення з переривання . . . . .                                   | 78        |
| <b>9 Висновки</b>                                                        | <b>79</b> |

## 1 Статус документа та архітектурна концепція

Цей документ є офіційним технічним описом та архітектурним стандартом 16-розрядного мікропроцесора **LIM M2**. Архітектура спроектована з фокусом на абсолютну передбачуваність, математичну точність і високу тактову частоту.

Ключовим рішенням у тракті даних LIM M2 стала **повна відмова від внутрішніх шин третього стану (Tri-State / Hi-Z)**. Внутрішні потоки даних побудовані на основі активних мультиплексорних дерев (Point-to-Point MUX Routing), що цілком усуває ризик конфліктів шини (bus contention), наскрізних струмів та паразитних ємнісних затримок.

### Нумерація системи команд

Система команд LIM M2 використовує суцільний 5-бітний діапазон базових опкодів: NOP=0, ADD=1, ..., MSP=26. Опкоди 27–31 зарезервовано для розширень наступного покоління.

### 1.1 Умовні позначення

| Позначення      | Архітектурне значення                                             |
|-----------------|-------------------------------------------------------------------|
| dst             | Цільовий регістр-приймач результату операції                      |
| src, srcA, srcB | Регістри-джерела вхідних операндів                                |
| imm             | Безпосереднє скалярне значення в тілі команди                     |
| addr            | Ефективна адреса пам'яті (16-бітна базова або 24-бітна розширена) |
| cfg             | 3-бітне поле конфігураційного модифікатора операції               |
| ZF, CF, OF      | Апаратні прапорці нуля, переносу та переповнення регістра FREG    |

### 1.2 Єдина філософія формату інструкцій

Архітектура процесора LIM M2 суворо дотримується універсальної парадигми: **спочатку завжди вказується цільовий приймач (куди), а потім вхідні операнди (звідки)**. Це забезпечує стрункий, логічний та інтуїтивно зрозумілий синтаксис:

- **Арифметичні операції (3 операнди):** ADD Rd, Rs1, Rs2 означає  $Rd \leftarrow Rs1 + Rs2$  (наприклад, ADD R0, R1, R2 обчислює  $R0 = R1 + R2$ ); аналогічно для SUB, MUL, DIV.
- **Логічні операції (2 операнди):** AND Rd, Rs означає  $Rd \leftarrow Rd \text{ AND } Rs$  (наприклад, AND R0, R1 обчислює  $R0 = R0 \& R1$ ); аналогічно для OR, NAND, NOR, XOR, XNOR.
- **Однобайтові унарні операції (1 операнд):** INC Rd ( $Rd \leftarrow Rd + 1$ ), DEC Rd ( $Rd \leftarrow Rd - 1$ ) та NOT Rd ( $Rd \leftarrow \sim Rd$ ). Працюють із єдиним цільовим регістром Rd на місці. Кодуються рівно в 1 байті ([7:3] опкод, [2:0] регістр Rd), другий байт із шини пам'яті не запитується.
- **Міжрегістрові пересилання:** LRR Rd, Rs означає пряме копіювання з точки Б (джерело Rs) у точку А (приймач Rd):  $Rd \leftarrow Rs$  (наприклад, LRR R0, R1 означає  $R0 = R1$ ).
- **Завантаження з пам'яті (LMR):** перший аргумент Rd визначає цільовий регістр процесора, куди записуються зчитані дані.

## 2 Архітектурний огляд процесора LIM M2

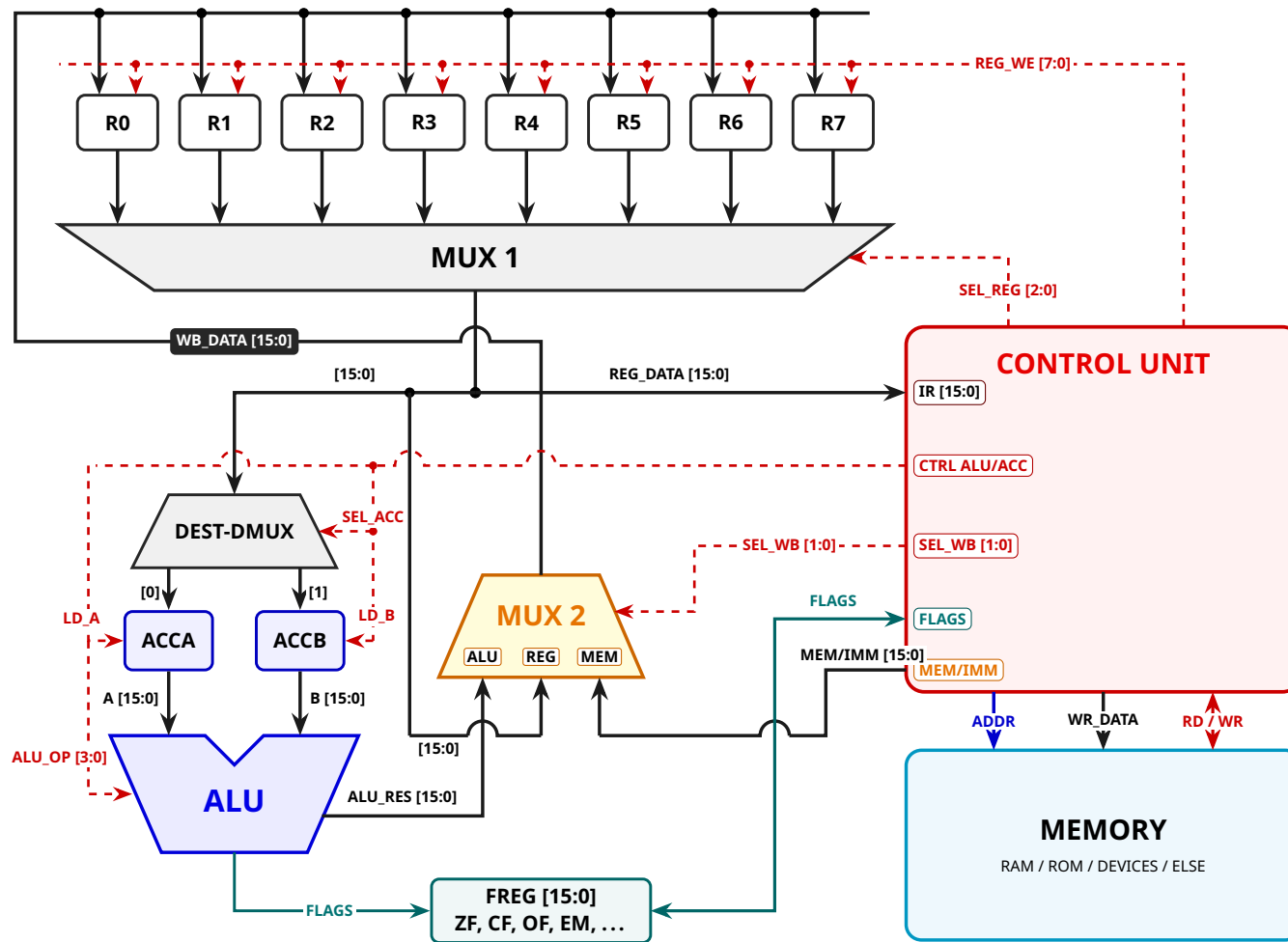
**LIM M2** — високопродуктивний 16-розрядний процесор із мікропрограмним керуванням, орієнтований на цілочисельні обчислення. Основні архітектурні переваги:

- **Чиста мультиплексорна маршрутизація:** Тракт даних використовує виключно активні

драйвери через каскад селекторів **MUX 1**, **DEST-DMUX** та **MUX 2**. Це гарантує ідеальну сумісність із кремнієвими технологіями (ASIC) та сучасними ПЛІС (FPGA).

- **Двоакумуляторний виконавчий вузол:** Регістри ACCA та ACCB надійно фіксують операнди перед АЛП, ізолюючи перехідні процеси від регістрового файлу.
- **Апаратне каскадування довгої арифметики (ALU Auto-Chaining):** Внутрішній тракт АЛП містить залишковий акумулятор (ALU\_RESIDUAL). У разі переносу ( $+1$  при сумі  $> 2^{16} - 1$ ), позики ( $-1$  при різниці  $< 0$ ) або надлишкових старших розрядів множення/ділення, залишок фіксується апаратно та автоматично додається чи віднімається під час наступної математичної операції. Це забезпечує наскрізну багатобайтову арифметику (32, 48, 64+ бітів) без окремих команд ADC/SBB. Скидання ланцюга виконується командою NOP.
- **Прямий канал пересилання (Reg-to-Reg Bypass):** Обхідний шлях із MUX 1 безпосередньо в MUX 2 дозволяє копіювати дані між регістрами за мінімальний час, не блокуючи АЛП.
- **Лінійна 24-бітна адресація пам'яті (до 16 МБ):** Спеціалізовані 24-бітні регістри PTREG (показник даних з автоінкрементом) та SREG (показник стека) дозволяють лінійно адресувати пам'ять та стек до 16 Мегабайтів.
- **Високий потенціал тактової частоти:** Завдяки відсутності ємнісного навантаження розподілених шин ядро розраховане на базову частоту **200 МГц**, а в оптимізованому кремнії досягає **700 МГц і вище**.

## 2.1 Концептуальна схема тракту даних



### 3 Програмно доступні регістри та стан процесора

Архітектурний стан процесора LIM M2 утворює чітку ортогональну систему регістрів:

| Регістр      | Розрядність | Призначення та ключові особливості                                                                                                                                       |
|--------------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>R0–R7</b> | 16 бітів    | Регістри загального призначення (GPR). Селектор MUX 1 забезпечує швидке зчитування; запис відбувається за індивідуальними стробами з шини WB_DATA.                       |
| <b>SREG</b>  | 24 біти     | Системний регістр стека (Stack Register). Забезпечує апаратну підтримку викликів процедур і кадрів стека з прямою 24-бітною адресацією стекового простору (до 16 Мбайт). |
| <b>PTREG</b> | 24 біти     | Високоточний регістр-показчик (Pointer Register). Забезпечує суцільну адресацію пам'яті до 16 Мбайт з апаратним автоінкрементом за прапорцем AIP.                        |
| <b>FREG</b>  | 16 бітів    | Регістр стану та керування (Processor Status & Control Register). Об'єднує арифметичні прапорці результату та біти конфігурації ядра.                                    |

#### 3.1 Карта регістра прапорців та керування (FREG)

Регістр FREG координує функціонування обчислювального ядра:

| Біт        | Мнемоніка | Функціональний опис                                                                                                            |
|------------|-----------|--------------------------------------------------------------------------------------------------------------------------------|
| <b>0</b>   | ZF        | Zero Flag: істинний при нульовому результаті операції.                                                                         |
| <b>1</b>   | CF        | Carry Flag: апаратний перенос або позика у знаковому/беззнаковому режимах.                                                     |
| <b>2</b>   | OF        | Overflow Flag: індикатор знакового переповнення додаткового коду.                                                              |
| <b>3</b>   | EM        | Error Math: апаратний прапорець математичних помилок (ділення на нуль).                                                        |
| <b>4–5</b> | CMP       | Compare Result: результат апаратного компаратора (00: скидання/не порівнювалось/помилка, 01: менше, 10: дорівнює, 11: більше). |
| <b>6–7</b> | CLK       | Clock Mode: конфігурація тактового генератора та дільників частоти.                                                            |
| <b>8</b>   | PIE       | Program Interrupt Enable: глобальний дозвіл програмних переривань.                                                             |
| <b>9</b>   | MIE       | Master Interrupt Enable: дозвіл зовнішніх апаратних переривань.                                                                |
| <b>10</b>  | DWR       | Disable Wait for Ready: форсований обмін без очікування сигналу готовності шини.                                               |
| <b>11</b>  | M24       | 24-bit Mode Enable: активація розширеного 24-бітного лінійного адресного простору.                                             |
| <b>12</b>  | AIP       | Auto-Increment PTREG: дозвіл апаратного автоінкременту показника PTREG.                                                        |
| <b>13</b>  | DBIT      | Displaced Bit: буфер витісненого біта зсувного вузла.                                                                          |
| <b>14</b>  | SMOD      | Shift Mode: керування циклічним режимом зсуву (0: заповнення нулями, 1: прохід через DBIT).                                    |
| <b>15</b>  | INTR      | In Interrupt: прапорець перебування у перериванні (при 1 нові переривання апаратно заблоковані).                               |

## 4 Формат машинних команд

Інструкції мікропроцесора LIM M2 мають ортогональний, модульний формат із детермінованим апаратним декодуванням. Базовий опкод завжди займає старші 5 бітів першого октету, що забезпечує миттєву диспетчеризацію в мікропрограмному керуючому ПЗП ядра.

### 4.1 Універсальний бітовий шаблон машинної команди

Кожна машинна інструкція починається з базового заголовка фіксованого розміру, за яким слідує динамічні операнди та аргументи:



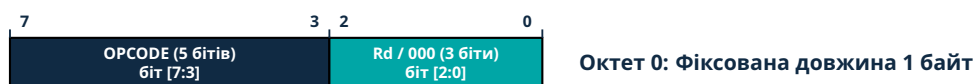
Молодші 3 біти першого октету ([2:0]) мають подвійне детерміноване призначення:

- В операціях з регістрами (арифметика, унарні команди, логіка, зсуви, пересилання, стек) біти [2:0] безпосередньо кодують **перший регістровий аргумент** (Rd або Rs1), що дозволяє адресувати повний набір із 8 регістрів R0–R7 без додаткових накладних витрат.
- У командах, які потребують вибору апаратного режиму чи селектора пам'яті/спецрегістрів (LMR, LRM, MSP, JMP, JCR, JNZ, JZ), біти [2:0] виступають конфігуратором режиму (CFG / MOD / SRC / DST).
- У безаргументних керуючих інструкціях (NOP, HALT, CLI, STI, RET) ці біти містять 000.

### 4.2 Клас I: Однобайтові інструкції (1 байт, 0 додаткових октет)

Інструкції керування (NOP, HALT, CLI, STI, RET) та унарні регістрові операції (NOT, INC, DEC) зчитуються за рівно один такт вибірки. Другий байт із шини пам'яті не запитується:

- Для безаргументних команд молодші 3 біти містять 000.
- Для унарних команд (INC Rd, DEC Rd, NOT Rd) молодші 3 біти ([2:0]) містять індекс цільового регістра Rd (R0–R7, єдиний аргумент).



### 4.3 Клас II: Трьохоперандні арифметичні інструкції (2 байти)

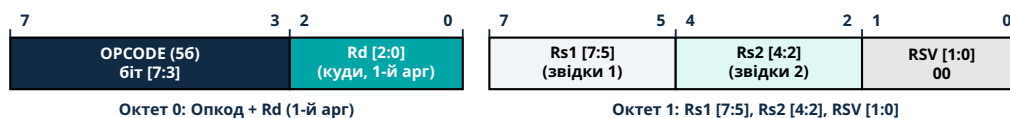
Команди обчислень (ADD, SUB, MUL, DIV, MOD) суворо реалізують архітектурну парадигму «спочатку приймач, потім доданки»:

$$\text{ADD Rd, Rs1, Rs2} \implies \text{Rd} \leftarrow \text{Rs1} + \text{Rs2} + \text{ALU\_RESIDUAL}$$

$$\text{MOD Rd, Rs1, Rs2} \implies \text{Rd} \leftarrow \text{Rs1} \pmod{\text{Rs2}}$$

Перший аргумент команди (цільовий регістр призначення Rd) розміщується безпосередньо в молодших 3 бітах першого октету, що дозволяє адресувати повні 8 регістрів R0–R7 для всіх трьох операндів:

- **Октет 0:** біти [7:3] — код операції (5 бітів); біти [2:0] — цільовий регістр призначення Rd (3 біти, R0–R7, перший аргумент).
- **Октет 1:** біти [7:5] — перший операнд-джерело Rs1 (3 біти, R0–R7); біти [4:2] — другий операнд-джерело Rs2 (3 біти, R0–R7); біти [1:0] — резерв (00).

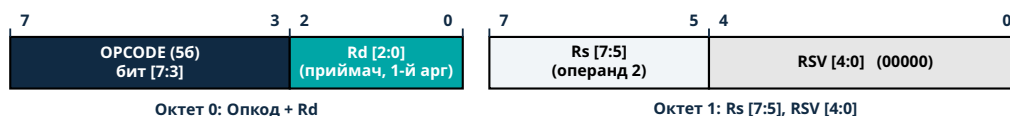


#### 4.4 Клас III: Двохоперандні логічні інструкції (2 байти)

Логічні операції (AND, OR, NAND, NOR, XOR, XNOR) мають рівно два операнди:

$$\text{AND } Rd, Rs \implies Rd \leftarrow Rd \text{ AND } Rs$$

- **Октет 0:** біти [7 : 3] — код операції; біти [2 : 0] — цільовий регістр-приймач Rd (R0–R7, перший аргумент).
- **Октет 1:** біти [7 : 5] — другий регістр-операнд Rs (R0–R7); біти [4 : 0] — зарезервовані (00000<sub>2</sub>).



#### 4.5 Клас IV: Міжрегістрове переміщення LRR (2 байти)

Інструкція LRR Rd, Rs копіює дані з джерела (точка Б, Rs) у приймач (точка А, Rd):

$$\text{LRR } Rd, Rs \implies Rd \leftarrow Rs$$

Переміщення здійснюється через прямий апаратний обхідний тракт **MUX 1** → **MUX 2** повз АЛП за один мікротакт, без зміни регістра прапорців FREG. Формат октетів: Октет 0 містить опкод 00111 та цільовий регістр Rd ([2:0]), Октет 1 містить регістр-джерело Rs ([7:5]) та резерв ([4:0] = 00000).

#### 4.6 Клас V: Апаратний компаратор CMP (2 байти)

Інструкція CMP Rs1, Rs2 порівнює значення регістрів без руйнування вмісту:

- **Октет 0:** код операції 01000, біти [2 : 0] задають перший операнд-джерело Rs1 (перший аргумент).
- **Октет 1:** біти [7 : 5] задають другий операнд-джерело Rs2; біти [4 : 0] містять 00000.

Результат порівняння фіксується у спеціалізованому 2-бітному апаратному полі регістра стану FREG. CMP (біти [5 : 4]):

| Значення FREG. CMP    | Умова               | Архітектурний зміст                           |
|-----------------------|---------------------|-----------------------------------------------|
| 01 (01 <sub>2</sub> ) | Rs1 < Rs2           | Перший операнд суворо менший за другий        |
| 10 (10 <sub>2</sub> ) | Rs1 = Rs2           | Значення операндів рівні                      |
| 11 (11 <sub>2</sub> ) | Rs1 > Rs2           | Перший операнд суворо більший за другий       |
| 00 (00 <sub>2</sub> ) | Вихідний / Скидання | Нічого не порівнювалося / помилка компаратора |

#### 4.7 Клас VI: Апаратний циклічний зсув на D-тригерах (BSL, BSR)

Команди зсуву BSL Rd, shift та BSR Rd, shift реалізовані у кремнії як **ітеративний 16-розрядний регістр зсуву на D-тригерах** (≈ 300 транзисторів замість ≈ 30 000 у матричного барельного вузла):

- Зсув виконується апаратно такт-за-тактом (for у залізі): зсув на 16 бітів потребує рівно на 15 тактів більше, ніж зсув на 1 біт.
- Октет 0: старші 5 бітів задають опкод (BSL: 11000, BSR: 11001), а молодші 3 біти ([2:0]) прямо вказують зсувний регістр Rd.
- Октет 1: весь байт ([7:0]) вказує кількість тактів зсуву shift (0–16).
- Поведінка бітів визначається регістром FREG: при SMOD = 1 увімкнено апаратну ротацію через витіснений біт; при SMOD = 0 порожні розряди заповнюються бітом із FREG.DBIT.



#### 4.8 Клас VII: Операції зі стеком (PUSH, POP)

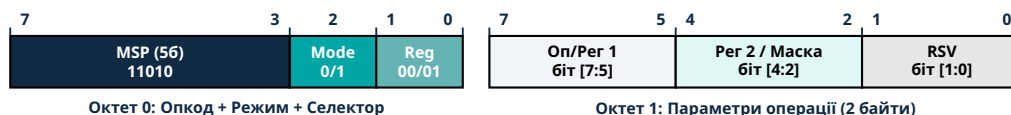
Команди стека зберігають регістр у пам'ять за 24-бітним вказівником SREG:

- Октет 0:** код операції (PUSH: 10110, POP: 10111), біти [2:0] прямо задають регістр операнда (Rs при збереженні, Rd при вилученні, перший аргумент).
- Октет 1:** біти [7:0] містять 00000000<sub>2</sub> (резерв).

#### 4.9 Клас VIII: Маніпуляція спеціальними регістрами (MSP)

Інструкція MSP забезпечує універсальну роботу з системними регістрами (FREG, PTREG та майбутні розширення):

- Октет 0:** біт [10] (біт 2 октету 0) задає режим:
  - 0: пряме завантаження значення у спецрегістр із регістрів загального призначення;
  - 1: функціональна модифікація спецрегістра (побітові маски або зміщення).
 Біти [9:8] (біти 1–0 октету 0) обирають цільовий спецрегістр:
  - 00<sub>2</sub>: регістр стану та прапорців FREG;
  - 01<sub>2</sub>: 24-бітний регістр-вказівник PTREG;
  - 10<sub>2</sub>, 11<sub>2</sub>: зарезервовано для майбутніх системних регістрів.
- Октет 1 (динамічний за CFG):**
  - При завантаженні** ([10] = 0): для FREG біти [7:5] задають регістр-джерело Rs; для PTREG біти [7:5] задають старший регістр Rs\_hi, а біти [4:2] — молодший регістр Rs\_lo.
  - При роботі з FREG** ([10] = 1, [9:8] = 00): біти [7:6] октету 1 задають побітову операцію (00=AND, 01=OR, 10=XOR, 11=XNOR), а біти [5:3] визначають регістр-маску Rmask.
  - При роботі з PTREG** ([10] = 1, [9:8] = 01): біти [7:6] октету 1 задають математичну операцію зміщення (00=ADD, 01=SUB), а біти [5:3] визначають рівно один регістр зміщення Rsrc (складені пари операндів заборонені). Додавання і віднімання використовують апаратний вузол АЛП із підтримкою прапорців переносу та позики.



#### 4.10 Клас IX: Інструкції розгалуження та переходів (2–3 байти)

Команди переходів (JMP, JCR, JNZ, JZ) підтримують три взаємовиключні режими адресації, що задаються полем CFG (біти [2:0] октету 0):

- **MOD = 0 (000): Безпосередня 16-бітна адреса (довжина 3 байти).** За октетом 0 слідує октети 1 та 2, що формують абсолютну 16-розрядну адресу цільової точки переходу в поточному 64-КБ сегменті.
- **MOD = 1 (001): Непрямий регістровий перехід (довжина 2 байти).** Октет 1 вказує цільовий регістр Rs (біти [7 : 5]), значення якого завантажується в PC.
- **MOD = 2 (010): Розширений 24-бітний перехід за вказівником PTREG.** Перехід здійснюється за фізичною 24-бітною адресою із системного регістра PTREG.

#### 4.11 Клас X: Інструкції обміну з пам'яттю LMR та LRM (2–4 байти)

Команди завантаження (LMR) та збереження (LRM) мають найвищий ступінь адаптивності. Поле конфігурації октету 0 ([2 : 0]) кодує:

- Біт [2]: розмір операнда SZ (0 = 8 бітів / байт, 1 = 16 бітів / слово).
- Біти [1 : 0]: джерело/приймач SRC / DST:
  - **0: Безпосередні дані (Inline Data):** 2 байти при SZ = 0 (байт за командою) або 3 байти при SZ = 1 (16-бітне слово за командою).
  - **1: Безпосередній вказівник (Imm Pointer):** 4 байти. Октет 1 задає робочий регістр, а октети 2–3 зберігають 16-бітну абсолютну адресу даних.
  - **2: Системний 24-бітний вказівник PTREG:** 2 байти. Октет 1 задає цільовий регістр. При активному прапорці FREG .AIP вказівник PTREG апаратно інкрементується за 0 додаткових тактів.
  - **3: Складена 24-бітна адреса з пари регістрів:** 2 байти. Октет 1 містить біти [7 : 5] (Arg1 / старші біти адреси) та біти [4 : 2] (Arg2 / молодші 16 бітів адреси). У команді LMR регістр Arg1 після читання перезаписується зчитаними даними.

## 5 Зведена таблиця команд ядра LIM M2

| №  | Код   | Мнемоніка | Клас        | Функціональне призначення                                                        |
|----|-------|-----------|-------------|----------------------------------------------------------------------------------|
| 0  | 00000 | NOP       | Керування   | Порожня операція та скидання залишкового регістра АЛП                            |
| 1  | 00001 | ADD       | Арифметика  | Додавання з автокаскадуванням переносу через залишок АЛП                         |
| 2  | 00010 | SUB       | Арифметика  | Віднімання з автокаскадуванням позики через залишок АЛП                          |
| 3  | 00011 | MUL       | Арифметика  | Множення з фіксацією старших розрядів у залишку АЛП                              |
| 4  | 00100 | DIV       | Арифметика  | Ділення з фіксацією залишку у внутрішньому регістрі АЛП                          |
| 5  | 00101 | LMR       | Пам'ять     | Завантаження з пам'яті/вводу в регістр (конфіг джерела 0–3, розмір 8/16 біт)     |
| 6  | 00110 | LRM       | Пам'ять     | Запис даних з регістра в пам'ять/вивід (конфіг призначення 0–3, розмір 8/16 біт) |
| 7  | 00111 | LRR       | Пересилання | Пряме міжрегістрове копіювання через обхідний MUX                                |
| 8  | 01000 | CMP       | Порівняння  | Апаратне порівняння без спотворення операндів                                    |
| 9  | 01001 | JMP       | Перехід     | Безумовний перехід (прямий, непрямий, 24-бітний)                                 |
| 10 | 01010 | JCR       | Перехід     | Умовний перехід за прапорцем переносу CF                                         |
| 11 | 01011 | JNZ       | Перехід     | Умовний перехід, якщо результат не нуль (ZF = 0)                                 |
| 12 | 01100 | JZ        | Перехід     | Умовний перехід за нульовим результатом (ZF = 1)                                 |

| №  | Код   | Мнемоніка | Клас       | Функціональне призначення                              |
|----|-------|-----------|------------|--------------------------------------------------------|
| 13 | 01101 | AND       | Логіка     | Побітове логічне множення (І)                          |
| 14 | 01110 | OR        | Логіка     | Побітове логічне додавання (АБО)                       |
| 15 | 01111 | NAND      | Логіка     | Побітове логічне І-НІ                                  |
| 16 | 10000 | NOR       | Логіка     | Побітове логічне АБО-НІ                                |
| 17 | 10001 | NOT       | Логіка     | Побітова інверсія операнда (доповнення до 1)           |
| 18 | 10010 | XOR       | Логіка     | Побітове виключне АБО                                  |
| 19 | 10011 | XNOR      | Логіка     | Побітова еквівалентність                               |
| 20 | 10100 | INC       | Арифметика | Апаратний інкремент значення на 1                      |
| 21 | 10101 | DEC       | Арифметика | Апаратний декремент значення на 1                      |
| 22 | 10110 | PUSH      | Стек       | Розміщення слова у стековому просторі                  |
| 23 | 10111 | POP       | Стек       | Вилучення вершини стека в цільовий регістр             |
| 24 | 11000 | BSL       | Зсуви      | Барельний зсув ліворуч із налаштуванням бітом          |
| 25 | 11001 | BSR       | Зсуви      | Барельний зсув праворуч із фіксацією в DBIT            |
| 26 | 11010 | MSP       | Система    | Маніпуляція службовими регістрами (FREG, PTREG)        |
| 27 | 11011 | MOD       | Арифметика | Обчислення залишку від цілочисельного ділення (modulo) |
| 28 | 11100 | RSV28     | Резерв     | Зарезервовано для розширеної адресації                 |
| 29 | 11101 | RSV29     | Резерв     | Зарезервовано для операцій із рухомою комою            |
| 30 | 11110 | RSV30     | Резерв     | Зарезервовано для апаратної криптографії               |
| 31 | 11111 | RSV31     | Резерв     | Зарезервовано для майбутніх розширень                  |

## 6 Детальна специфікація системи команд

Нижче подано повний архітектурний опис усіх 32 опкодів ядра LIM M2.

# NOP

00000

Керування

Апаратна порожня операція та скидання ланцюжка обчислень

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| Мнемоніка       | NOP                                                                                                               |
| Opcode          | 00000 (старші 5 бітів коду команди)                                                                               |
| Формат команди  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| Клас операції   | Керування                                                                                                         |
| Статус фіксації | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

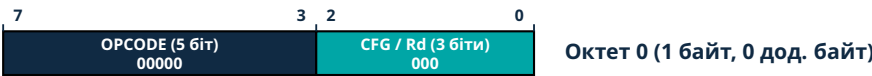
### Призначення інструкції

Апаратна порожня операція та скидання ланцюжка обчислень.

### Архітектурна семантика

Сигнатура: NOP  
Операція:  $PC \leftarrow PC + 1$ ,  $ALU\_RESIDUAL \leftarrow 0$ ,  $FREG.CF \leftarrow 0$ ,  $FREG.OF \leftarrow 0$   
Приклад: NOP ; Скидання ланцюжка переносу АЛП

### Конфігурація та операнди



Команда фіксованої довжини (1 байт). Операнди відсутні; поле cfg містить 000 (зарезервовано для калібрування затримок). Другий байт із шини пам'яті не запитується.

### Вплив на регістр прапорців (FREG)

Скидає прапорці переносу (CF), переповнення (OF) та позики; інші прапорці зберігаються.

### Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: TBD.

## Особливості реалізації та примітки

Виконується для тактової синхронізації конвеєра та апаратного скидання (Flush) внутрішнього залишкового регістра АЛП, розриваючи ланцюг автопереносу перед новими незалежними обчисленнями.

Поля з позначкою TBD визначають розширені параметри специфікації.

# ADD

00001

Арифметика

Додавання з автоматичним каскадуванням переносу

|                        |                                                                                                                   |
|------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Мнемоніка</b>       | ADD                                                                                                               |
| <b>Opcode</b>          | 00001 (старші 5 бітів коду команди)                                                                               |
| <b>Формат команди</b>  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| <b>Клас операції</b>   | Арифметика                                                                                                        |
| <b>Статус фіксації</b> | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

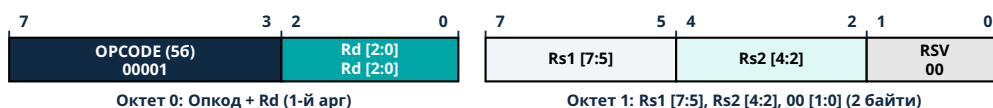
## Призначення інструкції

Додавання з автоматичним каскадуванням переносу.

## Архітектурна семантика

Сигнатура: `ADD arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 + arg2`  
 Операція: `sum <- arg1 + arg2 + ALU_RESIDUAL`  
`arg0 <- sum[15:0], ALU_RESIDUAL <- (sum > 0xFFFF ? 1 : 0)`  
 Приклад: `ADD R0, R1, R2 ; R0 = R1 + R2 (регістри R0–R7)`

## Конфігурація та операнди



**Октет 0:** код операції 00001, біти [2 : 0] задають цільовий регістр призначення Rd (R0–R7, перший аргумент).

**Октет 1:** біти [7 : 5] задають перший доданок Rs1 (R0–R7); біти [4 : 2] задають другий доданок Rs2 (R0–R7); біти [1 : 0] — резерв (00). Довжина команди: рівно 2 байти.

## Вплив на регістр прапорців (FREG)

Встановлює прапорець переносу CF та переповнення OF при перевищенні  $2^{16} - 1$ ; оновлює ZF за значенням dst.

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.

2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

### Особливості реалізації та примітки

Забезпечує наскрізну багатобайтову арифметику (32, 48, 64+ бітів). Якщо результат суми перевищує 16 бітів ( $> 2^{16} - 1$ ), перенос (+1) фіксується у внутрішньому регістрі АЛП та автоматично додається під час наступної операції ADD без потреби в окремих інструкціях ADC. Молодші 16 бітів детерміновано зберігаються в dst. Ланцюг скидається командою NOP або перезаписується новою математичною операцією.

# SUB

00010

Арифметика

Віднімання з автоматичним каскадуванням позики

|                        |                                                                                                                   |
|------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Мнемоніка</b>       | SUB                                                                                                               |
| <b>Opcode</b>          | 00010 (старші 5 бітів коду команди)                                                                               |
| <b>Формат команди</b>  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| <b>Клас операції</b>   | Арифметика                                                                                                        |
| <b>Статус фіксації</b> | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

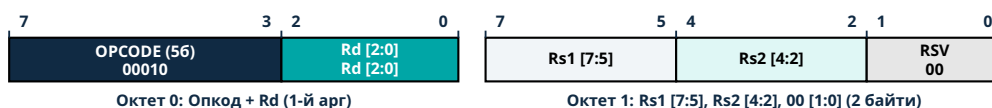
## Призначення інструкції

Віднімання з автоматичним каскадуванням позики.

## Архітектурна семантика

Сигнатура: SUB arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 - arg2  
 Операція: diff <- arg1 - arg2 - ALU\_RESIDUAL  
 arg0 <- diff[15:0], ALU\_RESIDUAL <- (diff < 0 ? 1 : 0)  
 Приклад: SUB R0, R1, R2 ; R0 = R1 - R2 (регістри R0–R7)

## Конфігурація та операнди



**Октет 0:** код операції 00010, біти [2 : 0] задають цільовий регістр призначення Rd (R0–R7, перший аргумент).

**Октет 1:** біти [7 : 5] задають зменшуване Rs1 (R0–R7); біти [4 : 2] задають від'ємник Rs2 (R0–R7); біти [1 : 0] — резерв (00). Довжина команди: рівно 2 байти.

## Вплив на регістр прапорців (FREG)

При різниці < 0 встановлює прапорець позики (Borrow / CF) та OF; оновлює ZF.

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).

3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

### Особливості реалізації та примітки

Реалізує неперервне багатослівне віднімання. При виникненні від'ємної різниці ( $< 0$ ) позика ( $-1$ ) зберігається у внутрішньому регістрі АЛП, і в наступній операції SUB АЛП автоматично віднімає 1 зі старшого слова. Результат молодшого слова детерміновано фіксується в dst. Для початку нової незалежної різниці ланцюг переривається інструкцією NOP.

# MUL

00011

Арифметика

Цілочисельне множення зі збереженням старшої частини в регістрі АЛП

|                        |                                                                                                                   |
|------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Мнемоніка</b>       | MUL                                                                                                               |
| <b>Opcode</b>          | 00011 (старші 5 бітів коду команди)                                                                               |
| <b>Формат команди</b>  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| <b>Клас операції</b>   | Арифметика                                                                                                        |
| <b>Статус фіксації</b> | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

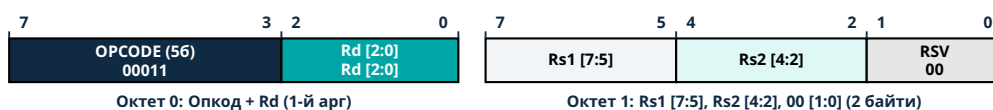
## Призначення інструкції

Цілочисельне множення зі збереженням старшої частини в регістрі АЛП.

## Архітектурна семантика

Сигнатура: `MUL arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 * arg2`  
 Операція: `prod <- arg1 * arg2`  
`arg0 <- prod[15:0], ALU_RESIDUAL <- prod[31:16]`  
 Приклад: `MUL R0, R1, R2 ; R0 = R1 * R2 (старша частина в ALU_RESIDUAL)`

## Конфігурація та операнди



**Октет 0:** код операції 00011, біти [2 : 0] задають цільовий регістр призначення Rd (R0–R7, перший аргумент).

**Октет 1:** біти [7 : 5] задають перший множник Rs1 (R0–R7); біти [4 : 2] задають другий множник Rs2 (R0–R7); біти [1 : 0] — резерв (00). Старші 16 бітів добутку автоматично фіксуються в ALU\_RESIDUAL. Довжина: 2 байти.

## Вплив на регістр прапорців (FREG)

Оновлює ZF; встановлює OF/CF, якщо старші розряди добутку (`prod[31:16]`) не дорівнюють 0.

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

## Особливості реалізації та примітки

При множенні молодші 16 бітів добутку надходять у цільовий регістр dst, а старші 16 бітів фіксуються у внутрішньому службовому регістрі АЛП (ALU Residual). У наступній операції додавання цей залишок автоматично підсумовується (апаратна підтримка MAC / довгих чисел). Старшу частину можна також отримати в регістр через додавання очищеного регістра з самим собою. Команда NOP очищує залишковий регістр АЛП.

# DIV

00100

Арифметика

Цілочисельне ділення зі збереженням залишку у внутрішньому регістрі АЛП

|                        |                                                                                                                   |
|------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Мнемоніка</b>       | DIV                                                                                                               |
| <b>Opcode</b>          | 00100 (старші 5 бітів коду команди)                                                                               |
| <b>Формат команди</b>  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| <b>Клас операції</b>   | Арифметика                                                                                                        |
| <b>Статус фіксації</b> | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

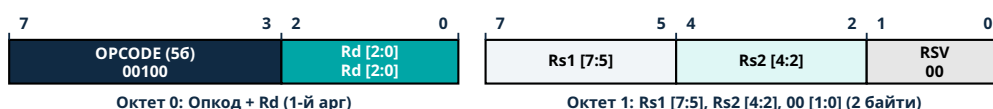
## Призначення інструкції

Цілочисельне ділення зі збереженням залишку у внутрішньому регістрі АЛП.

## Архітектурна семантика

Сигнатура: `DIV arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 / arg2`  
 Операція: `arg0 <- arg1 / arg2`  
`ALU_RESIDUAL <- arg1 % arg2`  
 Приклад: `DIV R0, R1, R2 ; R0 = R1 / R2 (остача modulo в ALU_RESIDUAL)`

## Конфігурація та операнди



**Октет 0:** код операції 00100, біти [2 : 0] задають цільовий регістр призначення Rd (R0–R7, перший аргумент).

**Октет 1:** біти [7 : 5] задають ділене Rs1 (R0–R7); біти [4 : 2] задають дільник Rs2 (R0–R7); біти [1 : 0] — резерв (00). Остача (modulo) автоматично фіксується в ALU\_RESIDUAL. Довжина: 2 байти.

## Вплив на регістр прапорців (FREG)

При діленні на нуль зводиться прапорець помилки EM (Error Math) без блокування ядра; оновлює ZF.

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

## Особливості реалізації та примітки

Результат цілочисельного ділення (частка) записується в цільовий регістр dst, а залишок від ділення (modulo) автоматично зберігається у внутрішньому службовому регістрі АЛП. Збережений залишок може бути задіяний у подальших операціях АЛП або скинутий командою NOP.

# LMR

00101  
Пам'ять

Завантаження даних із пам'яті та зовнішніх пристроїв у регістри

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| Мнемоніка       | LMR                                                                                                               |
| Opcode          | 00101 (старші 5 бітів коду команди)                                                                               |
| Формат команди  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| Клас операції   | Пам'ять                                                                                                           |
| Статус фіксації | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

## Призначення інструкції

Завантаження даних із пам'яті та зовнішніх пристроїв у регістри.

## Архітектурна семантика

Сигнатура: LMR arg0(REG), <SRC\_MODE> : arg0 = MEM[...]  
Режими: SRC=0: LMR R0, [PC+] ; Imm Data (1-2 байти)  
SRC=1: LMR R0, [[PC+]] ; Imm Pointer  
SRC=2: LMR R0, [PTREG] ; Системний показчик (AIP)  
SRC=3: LMR R0(R\_hi), R1(R\_lo) ; 24-біт адреса з пари  
Операція: arg0 <- MEM[EA] (старші біти адреси перезаписуються)  
Приклад: LMR R0, R1 ; EA=(R0«16)|R1, читання MEM[EA] в R0

## Конфігурація та операнди



**Октет 0:** біт [8] (SZ) задає розрядність (0 = 1 байт, 1 = 2 байти / слово); бити [10 : 9] (SRC) задають джерело даних (0 — дані безпосередньо за кодом команди, 1 — безпосередній показчик за командою, 2 — непрямо через 24-бітний показчик PTREG, 3 — непрямо через пару регістрів користувача).

**Октет 1:** бити [7 : 5] задають регістр Arg1 (R\_hi / приймач), бити [4 : 2] задають регістр Arg2 (R\_lo). У режимі SRC = 3 два 16-бітних регістри утворюють 32-бітне число, з якого для фізичної адресації використовуються 24 біти: Arg1 містить старші біти адреси, а Arg2 — молодші 16 біт. При цьому Arg1 одночасно виступає цільовим регістром: після вибірки з пам'яті старші біти адреси в ньому перезаписуються зчитаними даними.

## Вплив на регістр прапорців (FREG)

Усі прапорці регістра стану FREG зберігаються без змін.

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

## Особливості реалізації та примітки

Інструкція LMR є фундаментальним шлюзом введення зовнішньої інформації в регістровий файл процесора LIM M2 (*R0–R7*). Архітектура підтримує чотири ортогональні режими вибірки та повну 24-бітну адресацію:

1. У режимі  $SRC = 0$  процесор зчитує константу безпосередньо з потоку коду (1 або 2 байти) із просуванням лічильника команд PC.
2. У режимі  $SRC = 1$  за інструкцією зчитується абсолютний показник, за яким завантажуються дані.
3. У режимі  $SRC = 2$  читання виконується за 24-бітною фізичною адресою із системного регістра PTREG. За активного прапорця автоінкременту FREG.AIP (біт 12) регістр PTREG автоматично інкрементується на розмір вибірки (+1 або +2), забезпечуючи потокове DMA-подібне введення масивів за один такт.
4. У режимі  $SRC = 3$  користувач вказує два 16-бітних регістри (Arg1 та Arg2), які разом утворюють 32-бітне число, з якого на практиці адресації використовуються молодші 24 біти. Користувач визначає, що старші біти адреси знаходяться в Arg1, а молодші 16 біт адреси — в Arg2. У цьому режимі регістр Arg1 зі старшими бітами адреси також стає регістром призначення, куди зберігаються зчитані дані (тобто старші біти адреси в Arg1 перезаписуються даними, тоді як Arg2 зберігає молодші 16 біт адреси).

# LRM

Запис даних із регістрів у пам'ять та зовнішні пристрої

00110  
Пам'ять

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| Мнемоніка       | LRM                                                                                                               |
| Opcode          | 00110 (старші 5 бітів коду команди)                                                                               |
| Формат команди  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| Клас операції   | Пам'ять                                                                                                           |
| Статус фіксації | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

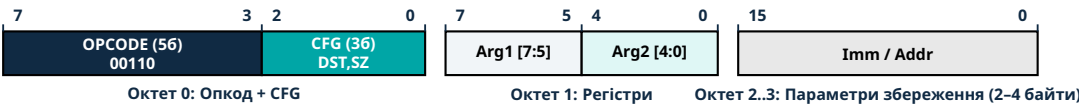
## Призначення інструкції

Запис даних із регістрів у пам'ять та зовнішні пристрої.

## Архітектурна семантика

Сигнатура: LRM arg0(REG), <DST\_MODE> : MEM[...] = arg0  
Режими: DST=0: LRM R0, [PC+] ; Вбудована пам'ять за кодом  
DST=1: LRM R0, [[PC+]] ; Imm Pointer  
DST=2: LRM R0, [PTREG] ; Системний покажчик (AIP)  
DST=3: LRM R0(R\_hi), R1(R\_lo) ; 24-біт адреса з пари  
Операція: MEM[EA] <- arg0[15:0] (або молодший байт при SZ=0)  
Приклад: LRM R0, R1 ; EA=(R0«16)|R1, запис R0 в MEM[EA]

## Конфігурація та операнди



**Октет 0:** біт [8] (SZ) задає розрядність (0 = 1 байт, 1 = 2 байти / слово); бити [10 : 9] (DST) задають цільовий приймач даних (0 — запис безпосередньо за кодом команди, 1 — запис за покажчиком за командою, 2 — непрямо через 24-бітний покажчик PTREG, 3 — непрямо через пару регістрів користувача).

**Октет 1:** бити [7 : 5] задають регістр-джерело Arg1 (Rs / R\_hi), бити [4 : 2] задають регістр Arg2 (R\_lo). У режимі DST = 3 два регістри формують 32-бітне число, з якого використовуються 24 біти адреси (Arg1 — старші біти адреси та джерело даних, Arg2 — молодші 16 біт). Знищення наявних даних за адресою запису є відповідальністю програміста.

## Вплив на регістр прапорців (FREG)

Усі прапорці регістра стану FREG зберігаються без змін.

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

## Особливості реалізації та примітки

Інструкція LRM є фундаментальним симетричним механізмом вивантаження регістрових даних процесора LIM M2 (*R0–R7*) в оперативну пам'ять та зовнішні порти периферії. Архітектура підтримує чотири ортогональні режими адресації:

1. У режимі  $DST = 0$  процесор записує 1 або 2 байти безпосередньо в пам'ять слідом за інструкцією (PC) зі зсувом лічильника команд. Режим застосовний для динамічних буферів та самомодифікації коду; перезапис даних лежить на відповідальності програміста.
2. У режимі  $DST = 1$  за інструкцією зчитується 16-бітний абсолютний адресний показник, за яким зберігаються дані.
3. У режимі  $DST = 2$  запис виконується за 24-бітною системною адресою регістра PTREG. За активного прапорця FREG.AIP (біт 12) регістр PTREG автоматично інкрементується на +1 або +2, забезпечуючи апаратне потокове DMA-виведення масивів у пам'ять без програмних циклів.
4. У режимі  $DST = 3$  використовується пара 16-бітних регістрів: Arg1 та Arg2 утворюють 32-бітне складене число, з якого молодші 24 біти задають фізичну адресу (Addr[23 : 0]). Дані з Arg1 записуються за обчисленою адресою, забезпечуючи прямий доступ до всього адресного простору 16 МБ.

# LRR

00111

Пересилання

Міжрегістрове переміщення (копіювання з точки Б в точку А)

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| Мнемоніка       | LRR                                                                                                               |
| Opcode          | 00111 (старші 5 бітів коду команди)                                                                               |
| Формат команди  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| Клас операції   | Пересилання                                                                                                       |
| Статус фіксації | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

## Призначення інструкції

Міжрегістрове переміщення (копіювання з точки Б в точку А).

## Архітектурна семантика

Сигнатура: LRR arg0(REG), arg1(REG) : arg0 = arg1  
Операція: arg0 <- arg1 (пряме копіювання з точки Б в точку А)  
Приклад: LRR R0, R1 ; R0 = R1 (значення R1 копіюється в R0)

## Конфігурація та операнди



**Октет 0:** код операції  $00111_2$  ( $0x07$ ), біти [2 : 0] задають цільовий регістр призначення Rd (R0–R7, точка А, перший аргумент).

**Октет 1:** біти [7 : 5] задають регістр-джерело Rs (R0–R7, точка Б); біти [4 : 0] містять  $00000_2$  (резерв). При виконанні значення Rs копіюється в Rd за 1 такт. Довжина команди: 2 байти.

## Вплив на регістр прапорців (FREG)

Прапорці зберігаються без змін.

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).

3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

### Особливості реалізації та примітки

Інструкція LRR є фундаментальною операцією пересилання даних між регістрами загального призначення процесора ( $R0-R7$ ). Вона здійснює передачу з джерела (точка Б) у приймач (точка А). Пересилання виконується через прямий апаратний обхідний тракт **MUX 1** → **MUX 2** в обхід АЛП, що виключає блокування обчислювального ядра та не змінює стан прапорців FREG.

# CMP

Апаратне порівняння операндів

01000

Порівняння

|                        |                                                                                                                   |
|------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Мнемоніка</b>       | CMP                                                                                                               |
| <b>Opcode</b>          | 01000 (старші 5 бітів коду команди)                                                                               |
| <b>Формат команди</b>  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| <b>Клас операції</b>   | Порівняння                                                                                                        |
| <b>Статус фіксації</b> | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

## Призначення інструкції

Апаратне порівняння операндів.

## Архітектурна семантика

Сигнатура: `CMP arg0(REG), arg1(REG) : compare(arg0, arg1)`  
 Операція: `if arg0 < arg1 then FREG.CMP <- 01b (менше)`  
`if arg0 == arg1 then FREG.CMP <- 10b (дорівнює)`  
`if arg0 > arg1 then FREG.CMP <- 11b (більше)`  
 (стан 00b — початкове скидання / не порівнювалось / помилка)  
 Приклад: `CMP R0, R1 ; 01: R0<R1, 10: R0==R1, 11: R0>R1 (00: скидання/помилка)`

## Конфігурація та операнди



**Октет 0:** код операції 01000, біти [2 : 0] задають перший операнд Rs1 (перший аргумент).  
**Октет 1:** біти [7 : 5] задають другий операнд Rs2; біти [4 : 0] містять 00000 (резерв).  
 Результат апаратного порівняння фіксується у полі `FREG.CMP` (біти [5 : 4]): 01 ( $R_{s1} < R_{s2}$ ), 10 ( $R_{s1} = R_{s2}$ ), 11 ( $R_{s1} > R_{s2}$ ), 00 (скидання / не порівнювалось / помилка). Значення регістрів не модифікуються. Довжина команди: 2 байти.

## Вплив на регістр прапорців (FREG)

Оновлює 2-бітне поле `FREG.CMP` (біти [5:4]), а також ZF, CF, OF без зміни регістрів.

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

## Особливості реалізації та примітки

Забезпечує швидке розгалуження алгоритмів.

# JMP

Безумовний перехід

01001

Перехід

|                        |                                                                                                                   |
|------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Мнемоніка</b>       | JMP                                                                                                               |
| <b>Opcode</b>          | 01001 (старші 5 бітів коду команди)                                                                               |
| <b>Формат команди</b>  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| <b>Клас операції</b>   | Перехід                                                                                                           |
| <b>Статус фіксації</b> | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

## Призначення інструкції

Безумовний перехід.

## Архітектурна семантика

Сигнатура: `JMP target(IMM|REG) : PC = target`

Операція: `PC <- target`

Приклад: `JMP R0 ; Безумовний перехід за адресою в R0`

## Конфігурація та операнди



**Октет 0:** код операції 01001, біти [2 : 0] (MOD): 000 — безпосередня 16-бітна адреса (довжина 3 байти); 001 — непрямий регістровий перехід (довжина 2 байти, октет 1 біти [7 : 5] задають Rs); 010 — 24-бітний перехід за показником PTREG.

## Вплив на регістр прапорців (FREG)

Прапорці зберігаються без змін.

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

### **Особливості реалізації та примітки**

Атомарне перезавантаження Program Counter за мінімум тактів.

# JCR

Умовний перехід за прапорцем переносу

**01010**

Перехід

|                        |                                                                                                                   |
|------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Мнемоніка</b>       | JCR                                                                                                               |
| <b>Opcode</b>          | 01010 (старші 5 бітів коду команди)                                                                               |
| <b>Формат команди</b>  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| <b>Клас операції</b>   | Перехід                                                                                                           |
| <b>Статус фіксації</b> | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

## Призначення інструкції

Умовний перехід за прапорцем переносу.

## Архітектурна семантика

Сигнатура: JCR target(IMM|REG) : if FREG.CF == 1 then PC = target

Операція: if FREG.CF == 1 then PC <- target

Приклад: JCR R2 ; Перехід за адресою R2 при наявності переносу

## Конфігурація та операнди



**Октет 0:** код операції 01010, біти [2 : 0] (MOD): 000 (16-біт адреса Imm, 3 байти), 001 (непрямо за Rs, 2 байти), 010 (24-біт за PTREG). Перехід виконується суворо при встановленому прапорці переносу FREG.CF = 1.

## Вплив на регістр прапорців (FREG)

Прапорці читаються без модифікації.

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

### **Особливості реалізації та примітки**

Фундаментальний примітив для реалізації довгої арифметики.

# JNZ

Умовний перехід, якщо не нуль

**01011**

Перехід

|                        |                                                                                                                   |
|------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Мнемоніка</b>       | JNZ                                                                                                               |
| <b>Opcode</b>          | 01011 (старші 5 бітів коду команди)                                                                               |
| <b>Формат команди</b>  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| <b>Клас операції</b>   | Перехід                                                                                                           |
| <b>Статус фіксації</b> | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

## Призначення інструкції

Умовний перехід, якщо не нуль.

## Архітектурна семантика

Сигнатура: JNZ target(IMM|REG) : if FREG.ZF == 0 then PC = target

Операція: if FREG.ZF == 0 then PC <- target

Приклад: JNZ R3 ; Перехід за адресою R3 при ZF == 0 (не нуль)

## Конфігурація та операнди



**Октет 0:** код операції 01011, біти [2 : 0] (MOD): 000 (16-біт адреса Imm, 3 байти), 001 (непрямо за Rs, 2 байти), 010 (24-біт за PTREG). Перехід виконується при скинутому прапорці нуля FREG.ZF = 0.

## Вплив на регістр прапорців (FREG)

Прапорці зберігаються без змін.

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

### **Особливості реалізації та примітки**

Мінімізований час ухвалення рішення в мікрокоді.

Поля з позначкою TBD визначають розширені параметри специфікації.

# JZ

Умовний перехід за нулем

01100

Перехід

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| Мнемоніка       | JZ                                                                                                                |
| Opcode          | 01100 (старші 5 бітів коду команди)                                                                               |
| Формат команди  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| Клас операції   | Перехід                                                                                                           |
| Статус фіксації | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

## Призначення інструкції

Умовний перехід за нулем.

## Архітектурна семантика

Сигнатура: JZ target(IMM|REG) : if FREG.ZF == 1 then PC = target

Операція: if FREG.ZF == 1 then PC &lt;- target

Приклад: JZ R4 ; Перехід за адресою R4 при ZF == 1 (нуль)

## Конфігурація та операнди



**Октет 0:** код операції 01100, біти [2 : 0] (MOD): 000 (16-біт адреса Imm, 3 байти), 001 (непрямо за Rs, 2 байти), 010 (24-біт за PTREG). Перехід виконується при встановленому прапорці нуля FREG.ZF = 1.

## Вплив на регістр прапорців (FREG)

Прапорці зберігаються без змін.

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

### **Особливості реалізації та примітки**

Оптимізовано для алгоритмів пошуку.

# AND

Побітове логічне І

01101  
Логіка

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| Мнемоніка       | AND                                                                                                               |
| Opcode          | 01101 (старші 5 бітів коду команди)                                                                               |
| Формат команди  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| Клас операції   | Логіка                                                                                                            |
| Статус фіксації | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

## Призначення інструкції

Побітове логічне І.

## Архітектурна семантика

Сигнатура: `AND arg0(REG), arg1(REG) : arg0 = arg0 & arg1`  
Операція: `arg0 <- arg0 AND arg1`  
Приклад: `AND R0, R1 ; R0 = R0 & R1`

## Конфігурація та операнди



**Октет 0:** код операції 01101, біти [2 : 0] задають цільовий приймач Rd (R0–R7, перший аргумент).

**Октет 1:** біти [7 : 5] задають другий операнд Rs (R0–R7); біти [4 : 0] містять 00000<sub>2</sub> (резерв). Довжина: 2 байти.

## Вплив на регістр прапорців (FREG)

Оновлює прапорець нуля ZF.

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.

4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).  
Точна сітка мікротактів та строби: **TBD**.

### Особливості реалізації та примітки

Базова операція для накладання бітових масок.

# OR

Побітове логічне АБО

01110  
Логіка

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| Мнемоніка       | OR                                                                                                                |
| Opcode          | 01110 (старші 5 бітів коду команди)                                                                               |
| Формат команди  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| Клас операції   | Логіка                                                                                                            |
| Статус фіксації | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

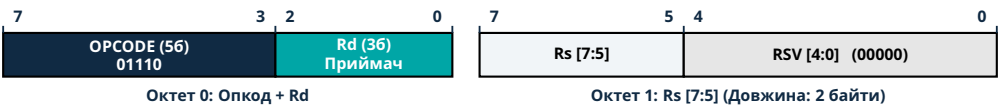
## Призначення інструкції

Побітове логічне АБО.

## Архітектурна семантика

Сигнатура: OR arg0(REG), arg1(REG) : arg0 = arg0 | arg1  
Операція: arg0 <- arg0 OR arg1  
Приклад: OR R0, R1 ; R0 = R0 | R1

## Конфігурація та операнди



**Октет 0:** код операції 01110, біти [2 : 0] задають цільовий регістр Rd (R0–R7, перший аргумент).  
**Октет 1:** біти [7 : 5] задають другий операнд Rs (R0–R7); біти [4 : 0] містять 00000<sub>2</sub> (резерв).  
Довжина: 2 байти.

## Вплив на регістр прапорців (FREG)

Оновлює прапорець ZF.

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

### **Особливості реалізації та примітки**

Швидке об'єднання бітів керування.

Поля з позначкою TBD визначають розширені параметри специфікації.

# NAND

Побітове логічне I-HI

01111  
Логіка

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| Мнемоніка       | NAND                                                                                                              |
| Opcode          | 01111 (старші 5 бітів коду команди)                                                                               |
| Формат команди  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| Клас операції   | Логіка                                                                                                            |
| Статус фіксації | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

## Призначення інструкції

Побітове логічне I-HI.

## Архітектурна семантика

Сигнатура: `NAND arg0(REG), arg1(REG) : arg0 = ~(arg0 & arg1)`  
Операція: `arg0 <- NOT (arg0 AND arg1)`  
Приклад: `NAND R0, R1 ; R0 = ~(R0 & R1)`

## Конфігурація та операнди



**Октет 0:** код операції 01111, біти [2 : 0] задають цільовий регістр Rd (R0–R7, перший аргумент).  
**Октет 1:** біти [7 : 5] задають операнд Rs (R0–R7); біти [4 : 0] містять 00000<sub>2</sub> (резерв). Довжина: 2 байти.

## Вплив на регістр прапорців (FREG)

Оновлює прапорець ZF.

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

### **Особливості реалізації та примітки**

Дозволяє синтезувати компактний двійковий код.

# NOR

Побітове логічне АБО-НІ

10000

Логіка

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| Мнемоніка       | NOR                                                                                                               |
| Opcode          | 10000 (старші 5 бітів коду команди)                                                                               |
| Формат команди  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| Клас операції   | Логіка                                                                                                            |
| Статус фіксації | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

## Призначення інструкції

Побітове логічне АБО-НІ.

## Архітектурна семантика

Сигнатура: `NOR arg0(REG), arg1(REG) : arg0 = ~(arg0 | arg1)`  
Операція: `arg0 <- NOT (arg0 OR arg1)`  
Приклад: `NOR R0, R1 ; R0 = ~(R0 | R1)`

## Конфігурація та операнди



**Октет 0:** код операції 10000, біти [2 : 0] задають цільовий регістр Rd (R0–R7, перший аргумент).  
**Октет 1:** біти [7 : 5] задають операнд Rs (R0–R7); біти [4 : 0] містять 00000<sub>2</sub> (резерв). Довжина: 2 байти.

## Вплив на регістр прапорців (FREG)

Оновлює прапорець ZF.

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

### **Особливості реалізації та примітки**

Розширює набір швидких перетворень.

# NOT

Побітова інверсія

10001

Логіка

|                        |                                                                                                                   |
|------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Мнемоніка</b>       | NOT                                                                                                               |
| <b>Opcode</b>          | 10001 (старші 5 бітів коду команди)                                                                               |
| <b>Формат команди</b>  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| <b>Клас операції</b>   | Логіка                                                                                                            |
| <b>Статус фіксації</b> | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

## Призначення інструкції

Побітова інверсія.

## Архітектурна семантика

Сигнатура: NOT arg0(REG) : arg0 = ~arg0

Операція: arg0 &lt;- NOT arg0

Приклад: NOT R0 ; R0 = ~R0 (1-байтова інструкція)

## Конфігурація та операнди



**Однобайтова інструкція (довжина: 1 байт).** Октет 0: біти [7 : 3] містять опкод 10001, біти [2 : 0] прямо задають цільовий регістр Rd (R0–R7). Другий байт із шини пам'яті не запитується.

## Вплив на регістр прапорців (FREG)

Оновлює прапорець ZF.

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: TBD.

## Особливості реалізації та примітки

Виконується за мінімальну кількість мікротактів без вибірки другого байта.

# XOR

Побітове виключне АБО

10010  
Логіка

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| Мнемоніка       | XOR                                                                                                               |
| Opcode          | 10010 (старші 5 бітів коду команди)                                                                               |
| Формат команди  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| Клас операції   | Логіка                                                                                                            |
| Статус фіксації | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

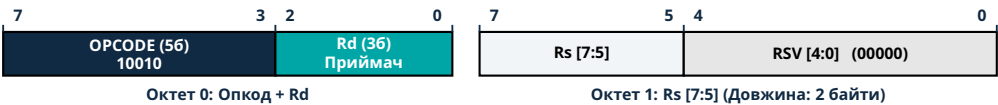
## Призначення інструкції

Побітове виключне АБО.

## Архітектурна семантика

Сигнатура: XOR arg0(REG), arg1(REG) : arg0 = arg0 ^ arg1  
Операція: arg0 <- arg0 XOR arg1  
Приклад: XOR R0, R1 ; R0 = R0 ^ R1 (при R0==R1 обнуляє R0)

## Конфігурація та операнди



**Октет 0:** код операції 10010, біти [2 : 0] задають цільовий регістр Rd (R0–R7, перший аргумент).  
**Октет 1:** біти [7 : 5] задають другий операнд Rs (R0–R7); біти [4 : 0] містять 00000<sub>2</sub> (резерв).  
Довжина: 2 байти.

## Вплив на регістр прапорців (FREG)

Оновлює прапорець ZF.

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

### **Особливості реалізації та примітки**

Основа для алгоритмів контрольних сум та шифрування.

# XNOR

Побітова еквівалентність

10011  
Логіка

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| Мнемоніка       | XNOR                                                                                                              |
| Opcode          | 10011 (старші 5 бітів коду команди)                                                                               |
| Формат команди  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| Клас операції   | Логіка                                                                                                            |
| Статус фіксації | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

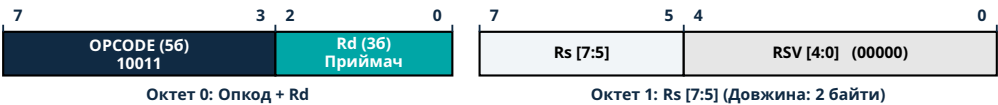
## Призначення інструкції

Побітова еквівалентність.

## Архітектурна семантика

Сигнатура:  $XNOR\ arg0(REG),\ arg1(REG) : arg0 = \sim(arg0 \wedge arg1)$   
Операція:  $arg0 \leftarrow NOT\ (arg0\ XOR\ arg1)$   
Приклад:  $XNOR\ R0,\ R1 ; R0 = \sim(R0 \wedge R1)$

## Конфігурація та операнди



**Октет 0:** код операції 10011, біти [2 : 0] задають цільовий регістр Rd (R0–R7, перший аргумент).  
**Октет 1:** біти [7 : 5] задають операнд Rs (R0–R7); біти [4 : 0] містять 00000<sub>2</sub> (резерв). Довжина: 2 байти.

## Вплив на регістр прапорців (FREG)

Оновлює прапорець ZF.

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

### **Особливості реалізації та примітки**

Швидка перевірка відповідності бітових масок.

Поля з позначкою TBD визначають розширені параметри специфікації.

# INC

Інкремент на одиницю

10100

Арифметика

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| Мнемоніка       | INC                                                                                                               |
| Opcode          | 10100 (старші 5 бітів коду команди)                                                                               |
| Формат команди  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| Клас операції   | Арифметика                                                                                                        |
| Статус фіксації | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

## Призначення інструкції

Інкремент на одиницю.

## Архітектурна семантика

Сигнатура: `INC arg0(REG) : arg0 = arg0 + 1`  
Операція: `arg0 <- arg0 + 1`  
Приклад: `INC R0 ; R0 = R0 + 1` (1-байтова інструкція)

## Конфігурація та операнди



**Однобайтова інструкція (довжина: 1 байт).** Октет 0: біти [7 : 3] містять опкод 10100, біти [2 : 0] прямо задають регістр Rd (R0–R7). Другий байт із шини пам’яті не запитується.

## Вплив на регістр прапорців (FREG)

Оновлює ZF. Переповнення контролюється АЛП.

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: TBD.

## Особливості реалізації та примітки

Швидкий крок лічильника без зайвої вибірки з шини.

# DEC

Декремент на одиницю

10101

Арифметика

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| Мнемоніка       | DEC                                                                                                               |
| Opcode          | 10101 (старші 5 бітів коду команди)                                                                               |
| Формат команди  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| Клас операції   | Арифметика                                                                                                        |
| Статус фіксації | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

## Призначення інструкції

Декремент на одиницю.

## Архітектурна семантика

Сигнатура: `DEC arg0(REG) : arg0 = arg0 - 1`  
Операція: `arg0 <- arg0 - 1`  
Приклад: `DEC R0 ; R0 = R0 - 1` (1-байтова інструкція)

## Конфігурація та операнди



**Однобайтова інструкція (довжина: 1 байт).** Октет 0: біти [7 : 3] містять опкод 10101, біти [2 : 0] прямо задають регістр Rd (R0–R7). Другий байт із шини пам'яті не запитується.

## Вплив на регістр прапорців (FREG)

Оновлює ZF. Позика контролюється АЛП.

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: TBD.

## Особливості реалізації та примітки

Компактні декрементні цикли без зчитування другого байта.

# PUSH

Збереження слова в стек

10110

Стек

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| Мнемоніка       | PUSH                                                                                                              |
| Opcode          | 10110 (старші 5 бітів коду команди)                                                                               |
| Формат команди  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| Клас операції   | Стек                                                                                                              |
| Статус фіксації | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

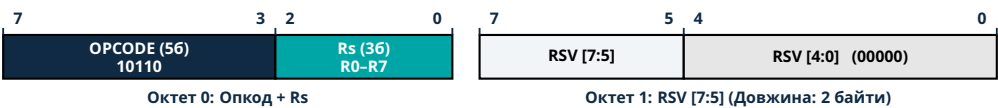
## Призначення інструкції

Збереження слова в стек.

## Архітектурна семантика

Сигнатура: PUSH arg0(REG) : SREG -= 2, MEM[SREG] = arg0  
Операція: SREG <- SREG - 2, MEM[SREG] <- arg0  
Приклад: PUSH R0 ; Зберегти слово R0 на вершину стека

## Конфігурація та операнди



**Октет 0:** код операції 10110, біти [2 : 0] прямо задають зберезуваний регістр-джерело Rs (R0–R7, перший аргумент).  
**Октет 1:** біти [7 : 0] містять 00000000<sub>2</sub> (резерв). Довжина команди: 2 байти.

## Вплив на регістр прапорців (FREG)

Арифметичні прапорці не змінюються.

## Мікрооперації виконавчого тракту

- Комутація мультиплексорів тракту даних для вибірки операндів.
- Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
- Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
- Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

### **Особливості реалізації та примітки**

Апаратна підтримка підпрограм та збереження контексту.

Поля з позначкою TBD визначають розширені параметри специфікації.

# POP

10111

Стек

Вилучення слова з вершини стека

|                        |                                                                                                                   |
|------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Мнемоніка</b>       | POP                                                                                                               |
| <b>Opcode</b>          | 10111 (старші 5 бітів коду команди)                                                                               |
| <b>Формат команди</b>  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| <b>Клас операції</b>   | Стек                                                                                                              |
| <b>Статус фіксації</b> | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

## Призначення інструкції

Вилучення слова з вершини стека.

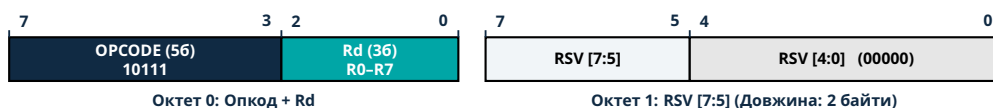
## Архітектурна семантика

Сигнатура: POP arg0(REG) : arg0 = MEM[SREG], SREG += 2

Операція: arg0 <- MEM[SREG], SREG <- SREG + 2

Приклад: POP R0 ; Витягнути слово з вершини стека в R0

## Конфігурація та операнди



**Октет 0:** код операції 10111, біти [2 : 0] прямо задають цільовий регістр-приймач Rd (R0–R7, перший аргумент).

**Октет 1:** біти [7 : 0] містять 00000000<sub>2</sub> (резерв). Довжина команди: 2 байти.

## Вплив на регістр прапорців (FREG)

Арифметичні прапорці не змінюються.

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

### **Особливості реалізації та примітки**

Симетричне відновлення контексту переривань.

# BSL

Апаратний зсув ліворуч (ітеративний регістр)

11000

Зсуви

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| Мнемоніка       | BSL                                                                                                               |
| Opcode          | 11000 (старші 5 бітів коду команди)                                                                               |
| Формат команди  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| Клас операції   | Зсуви                                                                                                             |
| Статус фіксації | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

## Призначення інструкції

Апаратний зсув ліворуч (ітеративний регістр).

## Архітектурна семантика

Сигнатура: BSL arg0(REG), arg1(IMM) : arg0 = arg0 « arg1  
Кодування: Октет 0: [7:3]=11000, [2:0]=Rd (цільовий регістр R0–R7)  
Октет 1: [7:0]=shift (кількість бітів зсуву 0–16)  
Операція: Послідовний тактовий зсув регістра Rd ліворуч на shift бітів:  
**Апаратний цикл (1 такт на 1 біт, ≈ 300 транзисторів):**  
Для кожного такту  $t \in [1..shift]$ :  
Старший біт 15 витісняється (last\_displaced <- bit 15);  
При SMOD = 1 у молодший біт 0 влітає витіснений біт (ротація);  
При SMOD = 0 у молодший біт 0 влітає біт із FREG.DBIT (0 або 1);  
Фіксація: FREG.DBIT <- last\_displaced (останній виштовхнутий біт).  
Приклад: BSL R0, 4 ; Зсув R0 ліворуч на 4 біти (виконується за 4 такти зсуву)

## Конфігурація та операнди



**Октет 0:** код операції 11000, біти [2 : 0] прямо задають зсуваний регістр Rd (R0–R7).  
**Октет 1:** весь байт ([7 : 0]) задає кількість тактів зсуву shift (0–16). Фізично зсув реалізується потактовим циклом на 16 D-тригерах. Довжина команди: 2 байти.

## Вплив на регістр прапорців (FREG)

Останній витіснений біт зберігається в FREG.DBIT (біт 13). Прапорець нуля FREG.ZF встановлюється при нульовому результаті. Керовано прапорцями FREG.SMOD (біт 14) та FREG.DBIT (біт 13).

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

## Особливості реалізації та примітки

Ітеративний зсувний регістр на D-тригерах: вартість усього  $\approx 300$  транзисторів замість  $\approx 30\,000$  для матричного одноктактового зсувача. Зсув на 16 бітів потребує на 15 тактів більше, ніж зсув на 1 біт.

# BSR

Апаратний зсув праворуч (ітеративний регістр)

11001  
Зсуви

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| Мнемоніка       | BSR                                                                                                               |
| Opcode          | 11001 (старші 5 бітів коду команди)                                                                               |
| Формат команди  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| Клас операції   | Зсуви                                                                                                             |
| Статус фіксації | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

## Призначення інструкції

Апаратний зсув праворуч (ітеративний регістр).

## Архітектурна семантика

Сигнатура: BSR arg0(REG), arg1(IMM) : arg0 = arg0 » arg1  
Кодування: Октет 0: [7:3]=11001, [2:0]=Rd (цільовий регістр R0-R7)  
Октет 1: [7:0]=shift (кількість бітів зсуву 0-16)  
Операція: Послідовний тактовий зсув регістра Rd праворуч на shift бітів:  
**Апаратний цикл (1 такт на 1 біт, ≈ 300 транзисторів):**  
Для кожного такту  $t \in [1..shift]$ :  
Молодший біт 0 витісняється (last\_displaced <- bit 0);  
При SMOD = 1 у старший біт 15 влітає витіснений біт (ротація);  
При SMOD = 0 у старший біт 15 влітає біт із FREG.DBIT (0 або 1);  
Фіксація: FREG.DBIT <- last\_displaced (останній виштовхнутий біт).  
Приклад: BSR R0, 4 ; Зсув R0 праворуч на 4 біти (виконується за 4 такти зсуву)

## Конфігурація та операнди



**Октет 0:** код операції 11001, біти [2 : 0] прямо задають зсуваний регістр Rd (R0-R7).  
**Октет 1:** весь байт ([7 : 0]) задає кількість тактів зсуву shift (0-16). Фізично зсув реалізується потактовим циклом на 16 D-тригерах. Довжина команди: 2 байти.

## Вплив на регістр прапорців (FREG)

Останній витіснений біт зберігається в FREG.DBIT (біт 13). Прапорець нуля FREG.ZF встановлюється при нульовому результаті. Керовано прапорцями FREG.SMOD (біт 14) та FREG.DBIT (біт 13).

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

## Особливості реалізації та примітки

Ітеративний зсувний регістр на D-тригерах: вартість усього  $\approx 300$  транзисторів замість  $\approx 30\,000$  для матричного одноктактового зсувача. Зсув на 16 бітів потребує на 15 тактів більше, ніж зсув на 1 біт.

# MSP

Маніпуляція спеціальними регістрами (FREG, PTREG)

11010

Система

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| Мнемоніка       | MSP                                                                                                               |
| Opcode          | 11010 (старші 5 бітів коду команди)                                                                               |
| Формат команди  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| Клас операції   | Система                                                                                                           |
| Статус фіксації | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

## Призначення інструкції

Маніпуляція спеціальними регістрами (FREG, PTREG).

## Архітектурна семантика

Сигнатура: MSP sreg, [op], args : робота зі спец-регістрами

Операція: **Пряме завантаження (біт 10 = 0):**

sreg=00 (FREG): FREG <- Rs[15:0] (октет 1: [7:5]=Rs)

sreg=01 (PTREG): PTREG <- (Rs\_hi << 16) | Rs\_lo (октет 1: [7:5]=Rs\_hi, [4:2]=Rs\_lo)

**Модифікація (біт 10 = 1):**

sreg=00 (FREG): FREG <- FREG  $\odot$  Rs (октет 1: [7:6]=op {AND,OR,XOR,XNOR}, [5:3]=Rs)

sreg=01 (PTREG): PTREG <- PTREG  $\pm$  Rs (октет 1: [7]=op {ADD,SUB}, [6:4]=Rs)

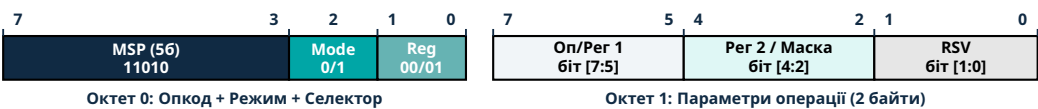
Приклад: MSP FREG, R0 ; Прямий запис у FREG значення R0

MSP PTREG, R0, R1 ; Завантаження 24-бітного PTREG = {R0, R1}

MSP FREG, OR, R2 ; Накладання бітової маски: FREG = FREG | R2

MSP PTREG, ADD, R3 ; Зсув покажчика: PTREG = PTREG + R3

## Конфігурація та операнди



**Окетт 0:** код операції 11010. Біт [10] (біт 2 октету 0) задає режим: 0 — пряме завантаження, 1 — модифікація. Біти [9 : 8] (біти 1–0 октету 0) задають селектор: 00 = FREG, 01 = PTREG.

**Окетт 1:** при завантаженні ([10] = 0) задає джерело: Rs для FREG або пару Rs\_hi, Rs\_lo для PTREG. При модифікації ([10] = 1) біти [7 : 6] кодують операцію (для FREG: AND, OR, XOR, XNOR; для PTREG: ADD, SUB), а біти [5 : 3] задають рівно один регістр операнда/маски Rs. Довжина команди: 2 байти.

## Вплив на регістр прапорців (FREG)

Прапорці FREG модифікуються напряму або маскою. Арифметика PTREG формує прапорець переносу CF через АЛП.

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

## Особливості реалізації та примітки

Універсальний керівний примітив системного рівня.

# MOD

Цілочисельний залишок від ділення (modulo)

11011

Арифметика

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| Мнемоніка       | MOD                                                                                                               |
| Opcode          | 11011 (старші 5 бітів коду команди)                                                                               |
| Формат команди  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| Клас операції   | Арифметика                                                                                                        |
| Статус фіксації | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

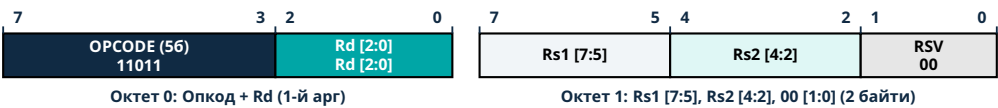
## Призначення інструкції

Цілочисельний залишок від ділення (modulo).

## Архітектурна семантика

Сигнатура: MOD arg0, arg1, arg2 :  $\text{arg0} = \text{arg1} \% \text{arg2}$   
Операція:  $\text{dst} \leftarrow \text{arg1} \% \text{arg2}$   
Приклад: MOD R0, R1, R2 ;  $\text{R0} = \text{R1} \% \text{R2}$  (залишок від ділення)

## Конфігурація та операнди



**Октет 0:** код операції 11011, біти [2 : 0] задають цільовий регістр Rd (R0–R7, перший аргумент).  
**Октет 1:** біти [7 : 5] задають ділене Rs1 (R0–R7); біти [4 : 2] задають дільник Rs2 (R0–R7); біти [1 : 0] — резерв (00). Довжина команди: 2 байти.

## Вплив на регістр прапорців (FREG)

При діленні на нуль зводиться апаратний прапорець помилки EM (Error Math) без зависання ядра; оновлює ZF.

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.

4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

### Особливості реалізації та примітки

Обчислює залишок від цілочисельного ділення діленого Rs1 на дільник Rs2 та записує результат безпосередньо в цільовий регістр Rd, не переписуючи проміжні результати в акумуляторах.

# RSV28

Апаратне розширення

**11100**

Резерв

|                        |                                                                                                                   |
|------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Мнемоніка</b>       | RSV28                                                                                                             |
| <b>Opcode</b>          | 11100 (старші 5 бітів коду команди)                                                                               |
| <b>Формат команди</b>  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| <b>Клас операції</b>   | Резерв                                                                                                            |
| <b>Статус фіксації</b> | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

## Призначення інструкції

Апаратне розширення.

## Архітектурна семантика

Сигнатура: RSV28 : зарезервовано

Операція: Reserved for LIM architecture hardware extensions

## Конфігурація та операнди



Зарезервовано для майбутніх версій архітектури LIM. Довжина: 1 байт.

## Вплив на регістр прапорців (FREG)

Не визначено.

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

## Особливості реалізації та примітки

Зарезервовано.

# RSV29

Апаратне розширення

11101  
Резерв

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| Мнемоніка       | RSV29                                                                                                             |
| Opcode          | 11101 (старші 5 бітів коду команди)                                                                               |
| Формат команди  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| Клас операції   | Резерв                                                                                                            |
| Статус фіксації | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

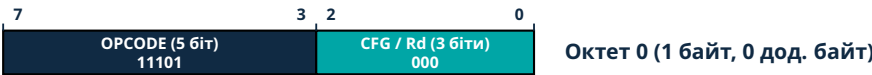
## Призначення інструкції

Апаратне розширення.

## Архітектурна семантика

Сигнатура: RSV29 : зарезервовано  
Операція: Reserved for LIM architecture hardware extensions

## Конфігурація та операнди



Зарезервовано для майбутніх версій архітектури LIM. Довжина: 1 байт.

## Вплив на регістр прапорців (FREG)

Не визначено.

## Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: TBD.

## Особливості реалізації та примітки

Зарезервовано.

Поля з позначкою TBD визначають розширені параметри специфікації.

# RSV30

Апаратне розширення

11110  
Резерв

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| Мнемоніка       | RSV30                                                                                                             |
| Opcode          | 11110 (старші 5 бітів коду команди)                                                                               |
| Формат команди  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| Клас операції   | Резерв                                                                                                            |
| Статус фіксації | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

### Призначення інструкції

Апаратне розширення.

### Архітектурна семантика

Сигнатура: RSV30 : зарезервовано  
Операція: Reserved for LIM architecture hardware extensions

### Конфігурація та операнди



Зарезервовано для майбутніх версій архітектури LIM. Довжина: 1 байт.

### Вплив на регістр прапорців (FREG)

Не визначено.

### Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: TBD.

## Особливості реалізації та примітки

Зарезервовано.

# RSV31

Апаратне розширення

11111  
Резерв

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| Мнемоніка       | RSV31                                                                                                             |
| Opcode          | 11111 (старші 5 бітів коду команди)                                                                               |
| Формат команди  | 5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode) |
| Клас операції   | Резерв                                                                                                            |
| Статус фіксації | Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації                                 |

### Призначення інструкції

Апаратне розширення.

### Архітектурна семантика

Сигнатура: RSV31 : зарезервовано  
Операція: Reserved for LIM architecture hardware extensions

### Конфігурація та операнди



Зарезервовано для майбутніх версій архітектури LIM. Довжина: 1 байт.

### Вплив на регістр прапорців (FREG)

Не визначено.

### Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних показників (PTREG, SREG).

Точна сітка мікротактів та строби: TBD.

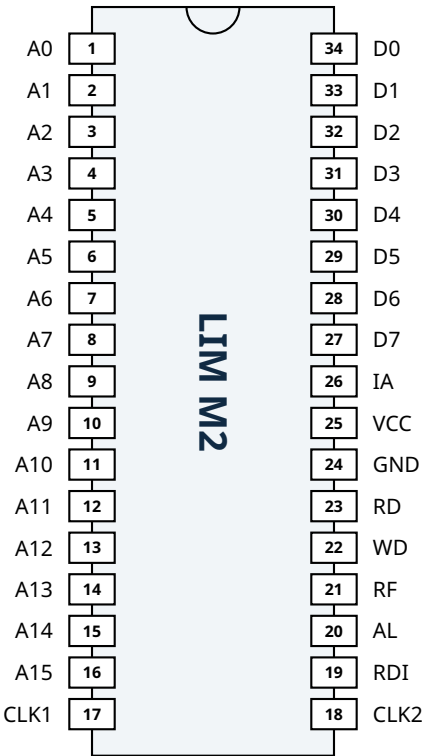
## Особливості реалізації та примітки

Зарезервовано.

Поля з позначкою TBD визначають розширені параметри специфікації.

## 7 Фізичний інтерфейс та цоколівка (34-вивідний корпус)

Процесор реалізовано у надійному компактному 34-вивідному корпусі з дворядним розташуванням:



### 7.1 Опис системних сигналів

| Вивід      | Тип               | Опис системного сигналу                                                                                                                             |
|------------|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| A0–A15     | Вихід / Tri-State | 16-розрядна адреса (мультиплексується зовнішнім стробом AL).                                                                                        |
| D0–D7      | Двоспрямований    | 8-розрядна зовнішня шина обміну з пам'яттю та периферією.                                                                                           |
| CLK1, CLK2 | Вхід              | Двофазна тактова синхронізація без перекриття фаз.                                                                                                  |
| VCC, GND   | Живлення          | Лінії живлення логіки процесора (+5 В номінально).                                                                                                  |
| RD, WD     | Вихід             | Строби зчитування (Read) та запису (Write) шини.                                                                                                    |
| AL         | Вихід             | Address Latch: строб фіксації адреси в зовнішньому регістрі.                                                                                        |
| RF         | Вхід              | Ready Flag: апаратне підтвердження готовності веденого пристрою.                                                                                    |
| RDI        | Вхід              | Raw Direct Interrupt: пріоритетний вхід апаратного переривання.                                                                                     |
| IA         | Вихід             | Interrupt Accepted / Interrupt Acknowledge: строб підтвердження прийому переривання; процесор переходить до зчитування номера вектора з шини даних. |

## 8 Підсистема переривань та векторна обробка

Підсистема переривань процесора LIM M2 спроектована для забезпечення мінімальної латентності реакції на події реального часу, строгого детермінізму та повного захисту від шинних конфліктів. Архітектура об'єднує апаратні зовнішні запити та програмні виключення в єдину 16-векторну ієрархію з надійним апаратним квітуванням на фізичному рівні.

### 8.1 Єдиний простір 16 векторів переривань

Процесор підтримує рівно 16 системних векторів (номери 0–15), які є ортогональними та спільними як для зовнішньої периферії, так і для внутрішніх подій процесорного ядра:

Table 3: Єдина таблиця 16 векторів переривань LIM M2

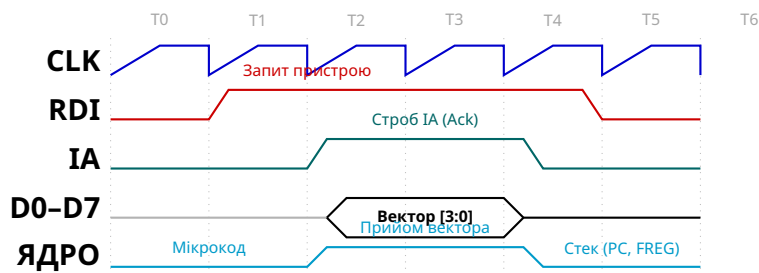
| Вектор          | Код       | Джерело / Тип         | Функціональне призначення                                         |
|-----------------|-----------|-----------------------|-------------------------------------------------------------------|
| <b>VEC 0</b>    | 0x00      | Апаратний / Скидання  | Power-on Reset: початкова ініціалізація процесора та регістрів.   |
| <b>VEC 1</b>    | 0x01      | Апаратний (NMI)       | Немасковане переривання: збій живлення, критична апаратна аварія. |
| <b>VEC 2</b>    | 0x02      | Програмна пастка      | Арифметичне виключення (ділення на нуль, прапорець FREG. EM).     |
| <b>VEC 3</b>    | 0x03      | Програмна пастка      | Системний виклик ядра (System Call / Trap диспетчера ОС).         |
| <b>VEC 4</b>    | 0x04      | Апаратний / Периферія | Системний таймер інтервалів / Квантувач планувальника.            |
| <b>VEC 5</b>    | 0x05      | Апаратний / Периферія | Контролер послідовного інтерфейсу (UART RX/TX).                   |
| <b>VEC 6</b>    | 0x06      | Апаратний / Периферія | Контролер прямого доступу до пам'яті (DMA Ready).                 |
| <b>VEC 7</b>    | 0x07      | Апаратний / Периферія | Зовнішній контролер переривань / Шинний міст.                     |
| <b>VEC 8–15</b> | 0x08–0x0F | Спільні (HW / SW)     | Користувацькі апаратні лінії та програмні вектори.                |

### 8.2 Апаратний протокол квітування (RDI → IA → D0–D7)

Для надійної взаємодії із зовнішніми контролерами реалізовано 4-фазний цикл апаратного рукоштовування (handshake), що повністю виключає необхідність постійного опитування регістрів (polling):

- Фаза запиту (RDI):** Периферійний пристрій встановлює активний високий рівень на вході RDI (*Raw Direct Interrupt*). Лінія опитується ядром на атомарних межах мікрокоманд.
- Фаза перевірки маски (Арбітраж):** Якщо глобальне маскування переривань дозволено (біт FREG. MIE = 1), процесор фіксує запит і завершує поточну неподільну мікрооперацію тракту даних.
- Фаза підтвердження (Строб IA):** Ядро генерує активний строб на виході IA (*Interrupt Accepted / Interrupt Acknowledge*, вивід 26). Цей строб сповіщає всі зовнішні пристрої на шині про готовність ядра прийняти номер вектора.
- Фаза передачі вектора (Шина D0–D7):** Одночасно з видачею IA зовнішня шина даних D0–D7 перемикається ядром у режим приймання (вхід), а лінія зчитування RD активується. Пристрій, що надіслав запит, утримує на лініях D0–D7 8-бітний ідентифікатор вектора (молодші 4 біти адресують вектори 0–15).

5. **Фаза фіксації та перехід:** Процесор фіксує дані з шини D0–D7, знімає сигнал IA та переходить до процедури збереження контексту.



### 8.3 Збереження контексту у 24-бітному стеку та вхід в обробник

Прийняття переривання дозволено виключно за умови, що процесор не перебуває в стані обробки іншого переривання ( $\text{FREG.INTR} = 0$ ) і переривання глобально дозволені ( $\text{FREG.MIE} = 1$ ). Якщо  $\text{FREG.INTR} = 1$ , будь-які повторні апаратні переривання апаратно заблоковані.

Під час фіксації переривання ядро виконує таку послідовність захисту стану:

1. **Збереження лічильника команд (PC):** Поточне значення PC зберігається на вершині системного стека за 24-бітним вказівником SREG:

$$\text{SREG} \leftarrow \text{SREG} - 2, \quad \text{Memory}[\text{SREG}] \leftarrow \text{PC}_{15:0}$$

2. **Збереження прапорців та статусу (FREG):** Поточний стан регістра FREG (з початковим значенням  $\text{FREG.INTR}=0$ ) розміщується у стеку:

$$\text{SREG} \leftarrow \text{SREG} - 2, \quad \text{Memory}[\text{SREG}] \leftarrow \text{FREG}$$

3. **Апаратне блокування вкладених переривань:** Ядро виставляє прапорець активності переривання  $\text{FREG.INTR} \leftarrow 1$  (біт 15) та скидає маску  $\text{FREG.MIE} \leftarrow 0$ , унеможливаючи каскадні повторні входи.
4. **Перехід за вектором:** Адреса обробника обчислюється на основі прийнятого номера вектора та завантажується в PC.

### 8.4 Повернення з переривання

Вихід з процедури обробки переривання здійснюється відновленням збереженого контексту зі стека:

1. Зі стека зчитується та відновлюється початковий стан регістра статусу:

$$\text{FREG} \leftarrow \text{Memory}[\text{SREG}], \quad \text{SREG} \leftarrow \text{SREG} + 2$$

(При цьому біт  $\text{FREG.INTR}$  автоматично повертається в 0, знімаючи блокування, а біт  $\text{FREG.MIE}$  відновлює свій попередній стан).

2. Зі стека відновлюється лічильник команд:

$$\text{PC} \leftarrow \text{Memory}[\text{SREG}], \quad \text{SREG} \leftarrow \text{SREG} + 2$$

3. Процесор продовжує виконання перерваного потоку інструкцій без жодних втрат даних.

## 9 Висновки

Архітектура **LIM M2** гармонійно поєднує перевірену надійність мікропрограмного керування та сучасну ефективність активного мультиплексорного тракту без третього стану. Завдяки усуненню шинних колізій та ємнісного розмивання фронтів процесор забезпечує виняткову цілісність сигналів і відкриває безпрецедентний частотний діапазон від 200 МГц до фізичних меж сучасного кремнію.